ServiceNow Platform Development

Project Documentation format

1. Introduction

Project Title: Script-Controlled ACL – Restrict Record Access Based on Field Value

Team Members: [List team members and their roles]

2. Project Overview

Purpose: To restrict access to records based on the value of a field (for example, Sensitivity = Confidential) using a script-controlled ACL, so that only authorised users can see or change sensitive records.

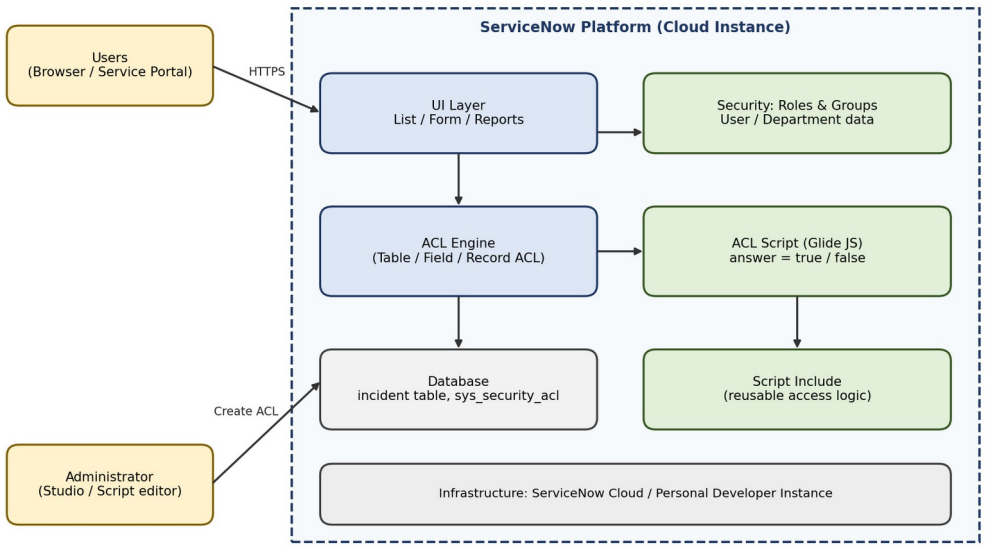
Features:

- Record-level ACL with a script that returns answer = true / false
- Field-value check combined with the logged-in user (assigned user, caller, privileged role)
- Same restriction in list view, form view and reports
- Read and write operation coverage
- Reusable access logic in a Script Include
- Deny by default when the field value is empty or unexpected

3. Architecture

- **Frontend:** ServiceNow UI (list view, form view, reports, Service Portal). No custom frontend code is needed; the ACL is applied by the platform.
- **Backend:** ACL script (Glide JavaScript) and a Script Include that holds the reusable access logic. The ACL engine evaluates roles, conditions and the script.
- **Database:** ServiceNow tables. The restricted table (e.g., incident) holds the Sensitivity field, and the ACL rules are stored in sys_security_acl.

Technical Architecture – Script-Controlled ACL on ServiceNow



4. Setup Instructions

- **Prerequisites:** A ServiceNow instance (Personal Developer Instance), an admin or security_admin role to create ACLs, test users with the itil role, and a custom Sensitivity field on the table.

Installation:

- Log in to the instance as an administrator and elevate to the security_admin role.
- Create the Sensitivity field (Public / Confidential) on the table and add test records.
- Create the test users and roles (assigned user, normal user, privileged user).
- Open System Security > Access Control (ACL) and create a new record ACL for the table (read).
- Paste the ACL script (see the example below), save and make sure the ACL is active.

Example ACL script (illustrative, test it in your own instance):

```
// Record ACL (advanced) – table: incident, operation: read
// Example names (u_sensitivity, u_confidential_viewer): change them to match your
instance
answer = false;
var sensitivity = current.getValue('u_sensitivity');
if (sensitivity == 'public') {
    answer = true;
} else if (gs.hasRole('u_confidential_viewer') ||
           current.getValue('assigned_to') == gs.getUserID() ||
           current.getValue('caller_id') == gs.getUserID()) {
    answer = true;
}
```

5. Folder Structure

- **Client:** Not applicable. The user interface is provided by the ServiceNow platform.
- **Server:** Platform records: Table field (u_sensitivity), ACL record (sys_security_acl), Script Include, test users, roles and groups. These can be grouped in one scoped application or an update set.

6. Running the Application

- Log in to the ServiceNow instance.
- **Frontend:** Open the table list or form (e.g., incident.list) as different users.
- **Backend:** No server start is needed; the ACL runs inside the platform whenever a record is requested. Use User Administration > Impersonate User to test each user.

7. API Documentation

The project adds no new API. The ACL is also applied when records are read through the ServiceNow REST Table API, which can be used for testing.

Method	Endpoint	Parameters	Example response
GET	/api/now/table/incident	sysparm_query, sysparm_limit	JSON list in "result" containing only the records the user is allowed to read
GET	/api/now/table/incident/{sys_id}	sys_id	The record, or an access error if the ACL denies it

8. Authentication

Authentication is handled by ServiceNow (user name and password, or SSO / OAuth if configured). Authorization is handled by roles, groups and the ACL script. For each request the ACL engine checks the roles and then runs the script; access is given only when the result is true. REST calls use the same ACLs for the authenticated user.

9. User Interface

[Add screenshots of the list view and form view as an authorised user and as an unauthorised user.]

10. Testing

Testing is done with user impersonation. The functional tests (FT-01 to FT-09), performance tests (PT-01 to PT-04) and UAT test cases (TC-001 to TC-012) are listed in the Project Development Phase documents. Both positive (allowed) and negative (denied) cases are covered.

11. Screenshots or Demo

[Add screenshots or a demo link after the ACL is built and tested.]

12. Known Issues

No issues have been recorded yet. Record any bugs found during UAT here.

13. Future Enhancements

- Apply the same pattern to other tables and fields
- Use ServiceNow security attributes or data conditions with the script
- Log denied access attempts for audit
- Add a message that tells the user why a record is restricted